TO COT OR NOT TO COT? CHAIN-OF-THOUGHT HELPS MAINLY ON MATH AND SYMBOLIC REASONING

Zayne Sprague♠**, Fangcong Yin**♠**, Juan Diego Rodriguez**♠**, Dongwei Jiang**♢**,**
Manya Wadhwa♠, **Prasann Singhal**♠, **Xinyu Zhao**♠,
Xi Ye♡, **Kyle Mahowald**♠, **Greg Durrett**♠

♠The University of Texas at Austin, ♢Johns Hopkins University, ♡Princeton University
zaynesprague@utexas.edu

ABSTRACT

Chain-of-thought (CoT) via prompting is the de facto method for eliciting reasoning capabilities from large language models (LLMs). But for what kinds of tasks is this extra “thinking” really helpful? To analyze this, we conducted a quantitative meta-analysis covering over 100 papers using CoT and ran our own evaluations of 20 datasets across 14 models. Our results show that CoT gives strong performance benefits primarily on tasks involving math or logic, with much smaller gains on other types of tasks. On MMLU, directly generating the answer without CoT leads to almost identical accuracy as CoT *unless* the question or model’s response contains an equals sign, indicating symbolic operations and reasoning. Following this finding, we analyze the behavior of CoT on these problems by separating planning and execution and comparing against tool-augmented LLMs. Much of CoT’s gain comes from improving symbolic execution, but it underperforms relative to using a symbolic solver. Our results indicate that CoT can be applied selectively, maintaining performance while saving inference costs. Furthermore, they suggest a need to move beyond prompt-based CoT to new paradigms that better leverage intermediate computation across the whole range of LLM applications [1] .

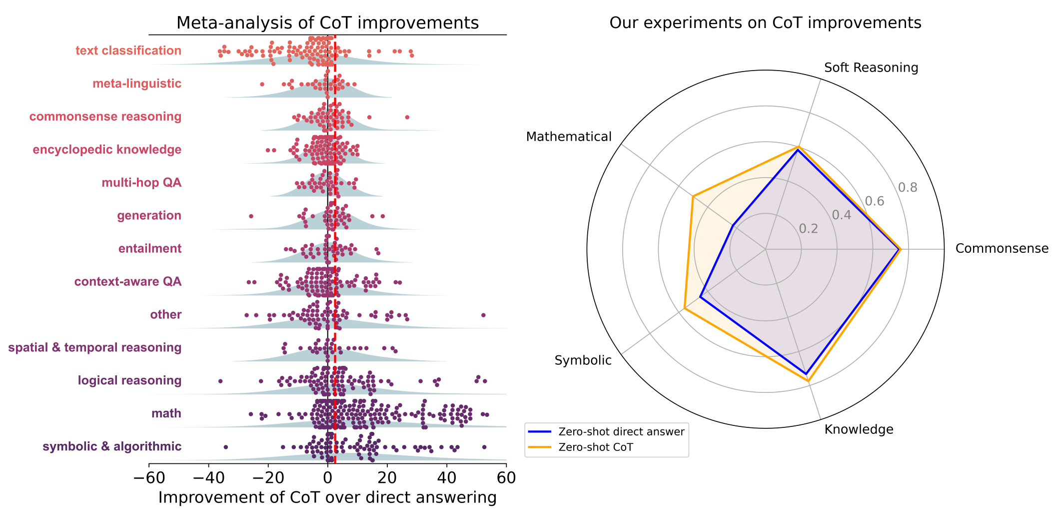

Figure 1: Left: meta-analysis of CoT literature; each point is a reported delta of CoT over direct answering for some (LLM, task) pair. Right: average performance of using zero-shot CoT v.s. direct answer prompts across five general reasoning categories, covering 20 datasets with 14 LLMs evaluated on each. In both sets of results, math and other kinds of symbolic reasoning are the domains that consistently see substantial improvements from CoT (red dotted line indicates the mean improvement from CoT across experiments).

[1]Our code can be found at `https://github.com/Zayne-sprague/To-CoT-or-not-to-CoT`.

1 INTRODUCTION

Chain-of-thought (CoT) (Nye et al., 2022; Wei et al., 2022) has become a widely used prompting technique for eliciting reasoning from language models. CoT can provide human-readable explanations of how problems are solved (Joshi et al., 2023; Lanham et al., 2023), but most frequently it is invoked to improve an LLM's ability to answer complex questions via intermediate computation (Madaan & Yazdanbakhsh, 2022; Wang et al., 2023a; Dziri et al., 2023). Current post-training schemes for LLMs heavily infuse CoT capabilities into models: systems like ChatGPT or Llama 3.1 default to CoT when given reasoning problems (OpenAI, 2023; Dubey et al., 2024).

CoT has seen widespread usage, but it is most heavily explored in the domain of mathematical reasoning (Zhou et al., 2023a; Fu et al., 2023; Chae et al., 2024; Xu et al., 2024b; Qi et al., 2024). In fact, many "reasoning" methods for LLMs are evaluated *only* in the math domain; for instance, Lightman et al. (2024) frame their paper as "complex multi-step reasoning" and Mixtral-Large2's release [2] cited effort "enhancing the model's reasoning capabilities", but performance is only reported on GSM8K and MATH. CoT is reported to be effective across a wide range of studies, but many of these studies focus on a narrow slice of the task space. In areas beyond math, results show that CoT is not as useful (Kambhampati et al., 2024a) or can even hurt performance (Wang et al., 2024).

In this work, we aim to evaluate where prompt-based CoT helps and why. We begin with a systematic meta-analysis of recent literature that reports performance of CoT versus direct answering (DA). We then augment this picture by conducting experiments on 20 datasets and 14 contemporary LLMs across zero-shot and few-shot prompt settings. **Finding 1: CoT only helps substantially on problems requiring mathematical, logical, or algorithmic reasoning.** Figure 1 shows this holds both across the literature and our own experiments. We find only a few cases of large gain in other kinds of tasks, and many of these outliers feature some component of symbolic reasoning. For instance, on MMLU (Hendrycks et al., 2021a) and MMLU Pro (Wang et al., 2024), we analyze the improvements from CoT and find that CoT *only* gives benefit on math slices of the dataset. **As much as 95% of the total performance gain from CoT on MMLU is attributed to questions containing "=" in the question or generated output.** For non-math questions, we find no features to indicate when CoT will help.

How can we better understand *why* CoT improves on these questions and only these questions? The math and formal logical reasoning datasets we consider can be broken down into two stages of processing: a planning step (e.g., parsing a problem into equations) and an execution step (building intermediate outputs and working towards a solution) (Ye et al., 2023; Wang et al., 2023b; Sun et al., 2024). **Finding 2: CoT primarily helps with the execution step that performs computation and symbolic manipulation, but falls short of what LLMs with tool augmentation can do.** We find that LMs prompted with CoT can generate executable formal solution plans and execute those plans better than direct answering. But using LMs to generate a solution plan and then using an external symbolic solver to solve the plan outperforms using CoT for both steps for these tasks.

These results paint a picture that CoT's utility is often circumscribed by tool augmentation: on problems where CoT helps, we already have more powerful tools than CoT that we can employ, and on "soft reasoning" problems like commonsense where no tools exist, we see limited benefit from CoT. This characterization has two major implications. First, CoT is unnecessary for many problems where it is widely employed: there exist more efficient prompting strategies that yield similar performance for much lower inference cost. Second, we see a critical need to move beyond prompt-based CoT to more sophisticated approaches based on search, interacting agents, or models more heavily fine-tuned for CoT. Future work can explore how intermediate computation can be better used to solve challenging problems outside of the math and symbolic reasoning domains.

2 BACKGROUND: CHAIN-OF-THOUGHT

The tasks we consider in this work consist of a question $\mathbf{q} \in \Sigma^*$ for a vocabulary Σ and an answer $a \in \mathcal{L}(\mathbf{q})$ for a label set $\mathcal{L}(\mathbf{q})$. $\mathcal{L}(\mathbf{q})$ can consist of a data type like boolean or integer, classification labels, or problem-dependent labels like names of entities from $\mathbf{q}$. One exception that we still

[2] `https://mistral.ai/news/mistral-large-2407/`

explore is BiGGen Bench (Kim et al., 2024), which instead relies on an LLM-as-a-judge (Dubois et al., 2023; Zheng et al., 2024b) to provide a label for generated long-form responses.

Prompting and chain-of-thought for reasoning A large language model places distributions over strings $p(\mathbf{y}) = \prod_{i=1}^{n} p_{\mathrm{LM}}(y_i)$ where $\mathbf{y} \in \Sigma^*$. In practice, we can interpret these as conditional distributions $p(\mathbf{y} \mid \mathbf{x})$ where $\mathbf{x}$ is a user's prompt. Typical invocation of an LLM involves forming a prompt $\mathcal{I}(\mathbf{q})$ that wraps the question with additional instruction, then drawing a sample response $\tilde{\mathbf{y}} \sim p(\mathbf{y} \mid \mathcal{I}(\mathbf{q}))$, and finally returning $a = \mathrm{extract}(\tilde{\mathbf{y}})$ using some kind of answer extractor.

For the tasks we consider in this work, the output $\tilde{\mathbf{y}}$ can take one of two forms. A **direct answer** only contains a string realization of a; e.g., $\mathbf{y} = (\_185, 4)$ which is detokenized as the answer $a = 1854$. A **chain of thought** is a longer sequence $\mathbf{y}$ including other tokens beyond the answer, e.g., $\mathbf{y} = (\_185, 6, \_\mathrm{minus}, \_2, \_\mathrm{equals}, \_185, 4)$. In both cases, the $\mathrm{extract}$ function must parse and detokenize the output; in CoT, there is some extra work to spot where the answer is placed.

Our prompts can explicitly encourage use of direct answer or chain of thought as strategies, which we denote as $\mathcal{I}_{\mathrm{da}}$ and $\mathcal{I}_{\mathrm{cot}}$. For eliciting CoT, this includes strategies like telling a model to "*think step by step*" (Kojima et al., 2022). For directly answering a question, a prompt may say "*immediately generate the answer*". We track the average location of the answer in the generated output for both CoT and direct prompts in Appendix F.3 to ensure that direct answer prompts give the answer early in the output. We also ensure that $\mathrm{extract}$ can parse answers from the generated output for each model, prompt, and dataset combination used in our experiments, tailoring the $\mathrm{extract}$ function as needed to ensure low unparseable rates for each model and task.[3] All prompts and outputs per dataset per model have been uploaded to Huggingface and we include examples of some of our prompts in the Appendix J. We also experiment with few-shot CoT prompts, which we find perform similarly to zero-shot prompts; details about these are given in Appendix E.

Symbolic reasoning Of key importance to this work is whether problems feature symbolic reasoning or not. We consider a problem to be **symbolic** if it can be grounded in a *natural, well agreed-upon* formal system. "12×4" is an example of a symbolic problem, which can be grounded in mathematics. Other systems include first-order logic (Saparov & He, 2023; Hua et al., 2024) or planning languages (Liu et al., 2023a; Valmeekam et al., 2023). Formally, for symbolic problems, we define a function f that acts as a map that produces some symbolic expression $\mathcal{S} = f(\mathbf{q})$ from the question. $\mathcal{S}$ can be used as input for a solver to derive an answer, $\hat{a} = \mathrm{solve}(\mathcal{S})$.

Conversely, a problem like *where on a river can you hold a cup upright to catch water on a sunny day?* from CommonsenseQA (Talmor et al., 2019) is **non-symbolic** by our definition. While this problem could be formalized with some kind of predicate logic (Zhou et al., 2022; Quan et al., 2024; Zhou et al., 2024) or grounded in some kind of physical simulation (Hao et al., 2023; Wong et al., 2023), there is not a natural nor well agreed-upon framework for solving it.

We view non-symbolic to symbolic reasoning as a spectrum. MuSR (Sprague et al., 2024) is a "semisymbolic" dataset in that it does contain an underlying formal system (e.g., for its murder mysteries portion, the notion that $\mathrm{motive}(X) \wedge \mathrm{means}(X) \wedge \mathrm{opportunity}(X) \implies \mathrm{murderer}(X)$), but also involves substantial commonsense reasoning that does not map onto a formal system. In these cases, we can still form $\mathcal{S} = f(\mathbf{q})$, but f must rely heavily on a language model and instantiate new information for $\mathcal{S}$ that is not directly represented in $\mathbf{q}$.

Central claim Figure 1 shows that there are a large number of positive results on CoT reported in the literature. Informally, we believe many readers of the literature to hold the following view: $\mathcal{I}_{\mathrm{cot}}$ ***will outperform*** $\mathcal{I}_{\mathrm{da}}$ ***on nearly all reasoning problems, whether those problems involve symbolic or non-symbolic reasoning.*** Our evidence does *not* support this conjecture. We will show that this performance boost is strongest for symbolic and semi-symbolic tasks, while giving little to no improvement (or even hurting performance) on non-symbolic tasks.

[3]We exclude a number of other "CoT-like" approaches in our analysis such as decomposed prompting (Khot et al., 2023; Zheng et al., 2024a) and multi-agent debate (Du et al., 2023; Chen et al., 2024). We focus on single prompt approaches. We deal with tool-augmented approaches in Section 5.